

Microprocessor Systems Final Project Report

LIDAR 3D MAPPING SYSTEM

Samarth Vijay

Instructors: Sharuk Athar, Thomas Doyle, Yasser Haddara

Department of Electrical and Computer Engineering, McMaster University

April 9th, 2025

COMP ENG 2DX3

Declaration of originality:

I certify that this is my own work, and it has not previously been submitted for any assessed qualification. I certify that the use of material from other sources has been properly and fully acknowledged in the text. I understand that the normal consequences of cheating in any element of an assessment, if proven and in the absence of mitigating circumstances, is that the class instructors' will be directed to fail the candidate in the assessment.

Signed: Samarth Vijay

Design Overview:

- The project design aims to create a low-cost, effective LIDAR sensor that is able to capture data and 3D Model open spaces such as hallways and containers
- The center of the design is the microcontroller, which we use the MSP-EXP432E401Y TI board with an ARM Cortex M4F microprocessor, with numerous GPIO ports used to control other devices, UART and I2C Protocols to communicate, as well as two onboard buttons along with 4 LEDs embedded on the controller.
- We use the ULN2003 driver board which is used to power a 28BYJ-48 stepper motor as well as a VL531X time of flight sensor used to capture data for the 3D modelling

Texas Instruments Microcontroller Features:

- Communication protocols including UART and I2C which are used to communicate through TOF sensor, and PC for visualization
- Max clock speed of 120MHz
- Specific bus speed is 12 MHz
- 1 MB Flash memory
- 256KB SRAM
- GPIO pins
- 2 buttons
- 4 LEDS
- JTAG Ethernet MicroUSB (Specifically for UART communication to PC)
- Programs and Executables written in C and Assembly Languages

VL531X TOF Sensor Features:

- 100Kbps communication speeds
- 3 Variable Distance Modes up to max range
- 50Hz Frequency
- 4m Range Distance
- 2.6-3.5V Operating Range

28BYJ-48 Stepper Motor Features:

- Full steps so 4 phases per step with 512 steps in 360 deg Rotation
- LED's representing each phase
- 5V-12V Operating Range

Serial Communication Methods:

- Using I2C communication protocol with 8 bit communication excluding 1 start bit and stop bit
- I2C specifically used between TOF sensor and Microcontroller, where TOF sensor is transmitting to TI board
- UART communication protocol used through COM12 and baud rate of 115200 bps
- UART specifically used between TI board and PC, where TI board is transmitting to PC for visualization

Visualization Method:

- Written in MATLAB, where UART information coming from the USB port from the microcontroller is redirected, and used for interpreting, conditioning and plotting data.

General Description:

This project is a cheaper and slightly less automated version of a LIDAR sensor where the TOF sensor is used to send data about the surrounding environment for modelling to your PC software.

The sensor is mounted onto the rotating end of a stepper motor, which rotates about the z-axis 360 degrees to collect XY plane data about the hallway or object its scanning, and has to be manually moved down a certain distance down the Z-axis before taking another 360 degree collection and scan. This will be repeated around 10-15 cycles of rotations and moved down incrementally until a fully visualized depth accurate model is plotted using MATLAB software.

The project was designed so that when the microcontroller is reset, the TOF sensor is automatically configured and activated, additionally providing UART output messages as it boots up. Next, PJ0 is used as the primary state function button where when the button is pressed initially the motor will begin to run, but without a button press the TOF sensor will simply be ready to capture data but not actually acquire anything. The PJ0 onboard button is used to toggle the motor function and TOF sensor data acquisition to be ON or OFF. A finite state machine implementation with an enumeration is used in the code design. Everything mentioned in this specific step is provided the system has not had any previous rotations and the motor is between 0 and 360 degrees of rotation.

Once the PJ0 button is pressed the motor rotates constantly, with the TOF sensor acquiring data at every 2.813 degrees by sending an Infrared Ray out of the sensor and calculating the amount of time it takes for the light to reflect off of an object and return to the sensor, and pausing for a

very brief period of time which allows for exactly 128 data point captures in one full stepper motor rotation.

Once a rotation is completed, there will be a printline through UART which tells the user a 360 degree rotation has been accomplished, and the motor will stop spinning.

For the purpose of untangling wires and maintaining an organized cable management, I decided to make PJ1 a homing function button, in which if pressed after a full rotation the stepper motor will complete a full 360 degree counterclockwise rotation to return the motor to its original position and prepare for another 360 degree capture of data at an increased value along the Z-axis. The data returned from the TOF sensor is in polar coordinates with a value and an angle returned in degrees for the motor.

During Distance Capturing, for each point data is sent to the microcontroller, and then sent from the microcontroller to the PC which then uses the MATLAB code to store and parse the transmitted data and polar coordinates for further calculations. The MATLAB code stores all 128 data points in an array to be used later.

Once all 128 points are registered into MATLAB software execution, each point is converted from polar coordinates to XY coordinates. The Z-axis is incremented every time an XY plane is developed fully with 128 data points on it to create a fully visualized model.

Block Diagram:

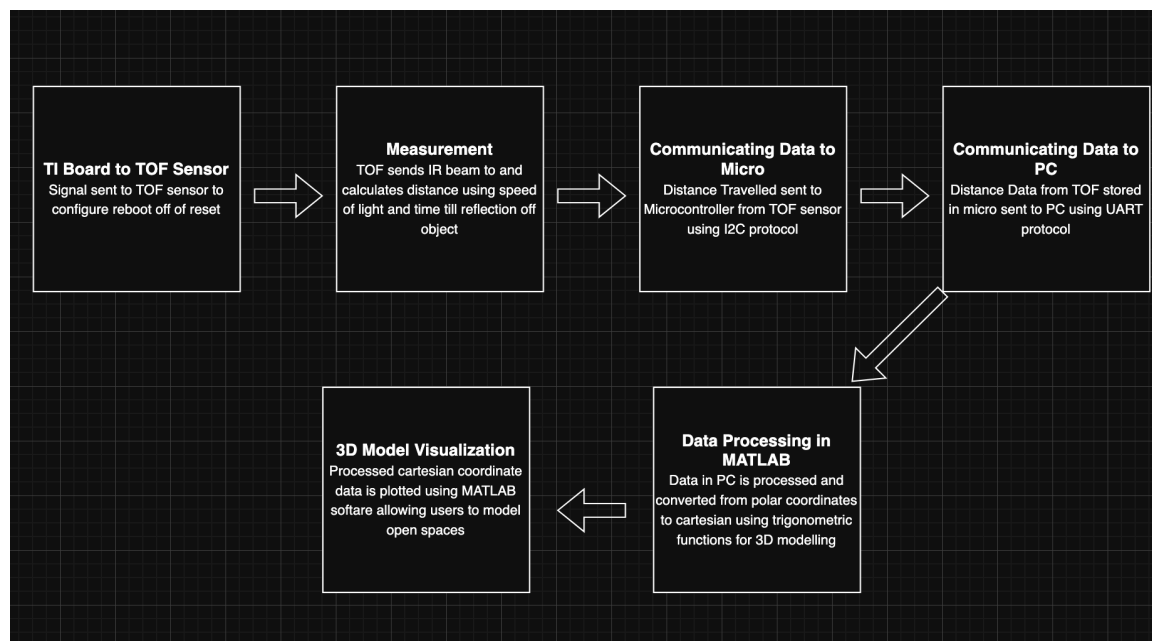


Figure 1: System Block Diagram

Device Characteristics Table:

MSP-EXP432E401Y	
Clock Speed	12 MHz
Baud Rate	115200 BPS
COM Port	COM12
Measurement LED	PF0
UART Print LED	PF1
Homing LED	PN0
Motor And TOF Sensor Activation Button	PJ0
Homing Button	PJ1

28BYJ-48 Stepper Motor	
V+	3.3V
V-	GND
IN1	PH0
IN2	PH1
IN3	PH2
IN4	PH3

VL53L1X Time of Flight	
VIN	3.3V
GND	GND
SCL	PB2
SDA	PB3

Table 1-3: Device Pin Assignments

Detailed Description:

Distance Measurement:

The VL53L1X Time-of-Flight sensor (TOF) is used by emitting an Infrared Ray of light from one of the two metal foils on it, and times how long the light ray takes to reflect off a surface of an object and return back to the sensor. By assuming the light ray from the sensor is moving at the speed of light, we can calculate the distance traveled by just the parameters of speed and 'time of flight'. The specific formula divides the value in half to get a singular distance measurement, because the time it takes to travel to an object and return back is twice the distance we are looking for.

$$\text{Time of Flight}/2 * C, \text{ where } C = \text{speed of light}.$$

Once the ray of light returns to the sensor, it goes through many processes before returning a digital signal to the microcontroller using the I2C protocol. It has to first transduce the signal, then condition the signal by shifting and amplifying it for accurate analog to digital conversion, and finally convert it to a digital value in 8 bits that is interpreted by the microcontroller through I2C. The system when executing the program must take a second for the TOF sensor to configure itself and boot up correctly and then waits for button press or further action to activate and actively capture and send data to the microcontroller. Similar to the trigger of the TOF sensor activation, the entire program is designed with polling methods of action rather than an interrupt driven system. The entire capturing data and booting the TOF sensor occurs within the C program code in the KEIL project file.

Once the sensor is booted and ready for data capture, there is an enumeration in the code that applies a finite state machine logic, where when the onboard button PJ0 is pressed, there is a variable **state** that is toggled as 0 for OFF and 1 for ON. Then there is a void function that updates the state of the system which includes the stepper motor and also the TOF data collection code. When state is set as 0, the enumeration is considered **IDLE**, and when the state is 1 the enumeration sets it to **RUNNING**. Then the function checks if the motor state is 0 or 1, and only calls the *motor_spin* function that rotates the motor one full step per function call if the motor state is **RUNNING**. When this function is polled in an infinite while loop in the main function, this allows the motor and TOF to continuously spin and collect data as long as the PJ0 button is not pressed again and the state is not toggled to OFF.

The program is set to collect 128 points of TOF data for a 360 degree rotation, where there are 512 steps in one full cycle. This means that for each phase of one step (there are 4 phases per

step which is 11.25 degrees of rotation), the TOF is taking a momentary delay and collecting data. This essentially means for every 2.83 degrees, the TOF collects one data point. Each GPIO pin of the ULN2003 driver is toggled one by one from PH0-3, allowing each phase to activate inside the stepper motor.

Each time a point of data is collected, the I2C protocol transmits the data from the sensor to the microcontroller, which is processed and using UART communication protocol print functions, is sent via USB port from the microcontroller to the PC for display. All TOF distance measurements, collections and boot configuration functions are used in the project code from the VL531X core API all found in the UM2356 manual. This was provided to us by the 2DX3 course files. Throughout the process of collecting TOF data, the program also has a variable **count** that tracks the number of steps until 512 of the motor, and multiplies the step count with 2.83 degrees to keep track of both the TOF distance measurement and the degree the motor is turned to at a specific measurement.

For example if a measurement of 300cm was taken after 16 increments of count, the PC display would show a value of 300cm for distance and an angle of 45 degrees.

At the end of each 360 degree rotation, the program enters another while loop, which only allows the program to exit when button PJ1 detects a press **and also** the stepper motor completes a full 360 degree rotation in the counterclockwise direction. When the onboard button is pressed, a message is displayed on the PC from UART that says 'home time'. A flag variable is initialized and used in the main function to allow the data acquisition process to repeat for another full rotation, only if the stepper motor is at its home position of 0 degrees. It does this by decrementing the count variable from 512 after every motor_spin function call in the opposite direction when button PJ1 is pressed.

PF1 LED is configured to blink every time UART communicates or sends a print line of the TOF data to the PC, while PF0 is configured to blink every time the TOF communicates data via I2C to the microcontroller. It is difficult to see but there is a slightly noticeable delay between the two LED's blinking. PN0 is configured as my additional status LED, which flashes every time the button PJ1 is pressed indicating it is returning to home position correctly.

Visualisation:

Using UART serial communication from the USB port to the microcontroller board, the MATLAB program is used to store, process, and plot the information from the TOF sensor after every 360 degree rotation. Using the baud rate of 115200 with a small time delay at the beginning of the program, it starts with displaying available ports for communication and specifically scanning COM12 port data for my PC for processing. After this, there is the

initialization of various arrays and variables for example **newDataArray** that parse the UART strings communicated and storing specifically the degree value correlated to each distance measurement from the sensor. There is also a variable called **numRotations** that defines how many 360 degree rotations the user would like to measure and plot for the 3D model. Based on numRotations, a for loop is run to convert each array of polar coordinates to cartesian, increment the Z-axis depth displacement value, and also create a new array to store the next array of data from the next cycle. Once all the data is collected, the number of arrays (based on numRotations) will be parsed into X and Y variable arrays, and for each array map the incrementing displacement to it. Each increment is set as whole numbers in the plot, but are considered as 20 cm per unit of depth graphed. This will allow each array that correlates to a certain 360 degree rotation, and will plot a plot plane on the XY axis on a certain Z-axis value for depth.

The main bulk of processing comes from converting from polar coordinates to cartesian and it is done by using the formulas below for each point out of 128 that is collected by the TOF sensor.

$$Y = distance * \sin(angle\ of\ distance)$$
$$X = distance * \cos(angle\ of\ distance)$$

An example of this if the sensor returned a value 200cm at 45 degrees, the MATLAB code would return an X coordinate of $200\text{cm} * \sin(45)$ which is equal to the Y coordinate value which is 141.42cm which is the point (141.42, 141.42).

Applications of LIDAR Technology:

1. Augmented Reality
 - a. LIDAR sensors can be used to model rooms and spaces
 - b. Can be used to simulate renovations or for entertainment purposes
2. Autonomous Vehicles
 - a. Object collision detection can be used for vehicles using LIDAR sensors
 - b. Automated parking and self-driving technology can implement these sensors with machine learning

Instructions:

1. Connect the TI board via USB port to your PC
2. Wire the circuit using the configurations and pin assignments in tables 1, 2 and 3 as well as the circuit diagram in figure 4
3. Mount the TOF sensor onto the rotating part of the stepper motor

4. Open the MATLAB script file, and comment out the displayPorts line in the first 5 lines if you know what port on your PC for communication you are using
5. If not, run the program to determine what port to use and set the port value based on the index of the array of port values your specific communication port is
6. Set numRotations in the MATLAB code to the number of 360 degree rotations you would like
7. Once your communication port is initialized and set in the code, Open the KEIL project
8. Open “options for target” in the menu on top, and ensure that optimization is set as -O0
9. Build, translate and load the program onto the microcontroller
10. Hit the onboard reset button
11. Wait for onboard LEDs to flash twice, then you are ready to start the collection process
12. Ensure that the MATLAB code is executed and running prior to starting data collection
13. Click PJ0 onboard button to start data collection and observe the values of distance and degrees in the MATLAB terminal
14. Once a 360 degree rotation is complete, it should display a message telling you it is done in MATLAB terminal
15. Click PJ1 onboard button to untangle wires and return stepper motor to original position
16. A message should display “home time!” in the MATLAB terminal
17. Increment the displacement of your system physically upwards after every rotation
18. Once all rotations are complete, a plot should appear in your MATLAB application which represents Y as your vertical, X as your horizontal and Z as your depth/displacement values with the number of planes on the plot matching numRotations
- 19. ENSURE THAT THE WIRES CONNECTED FROM YOUR TOF SENSOR DO NOT GO IN FRONT OF THE SENSOR WHILE IT IS COLLECTING DATA AND ROTATING**

The output for a hallway with 10 as numRotations would look like the image below. There were some slight discrepancies explained in point 18 of the instructions that contribute to how the model was plotted.

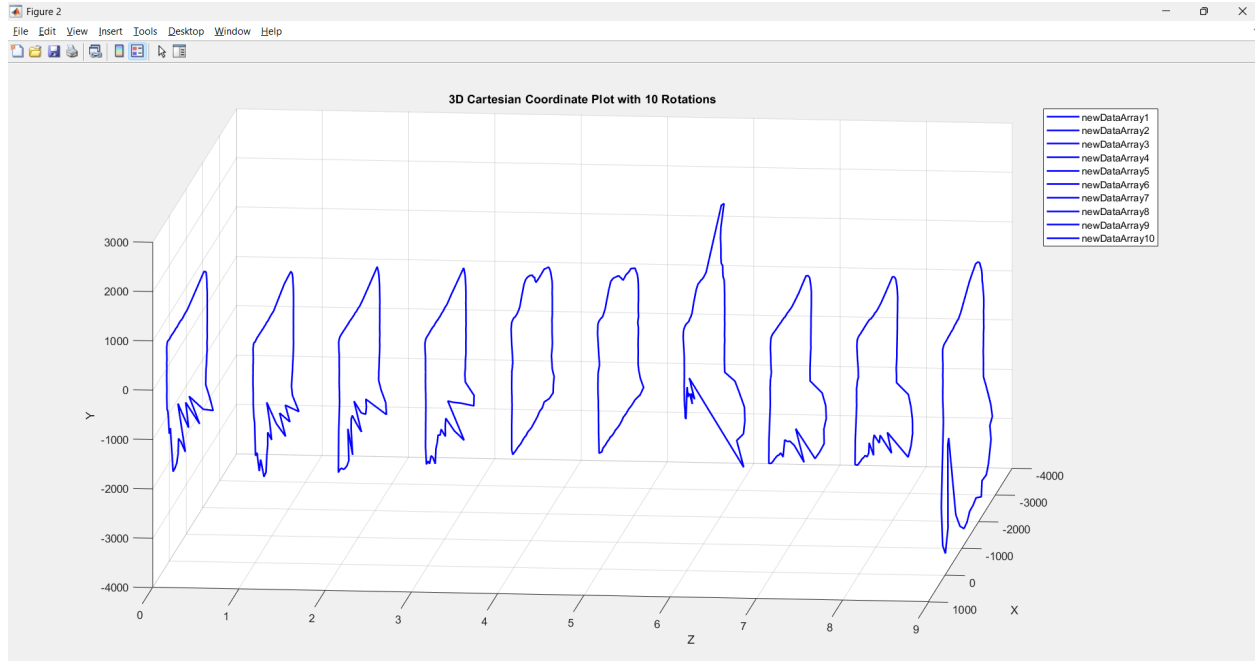


Figure 3: Example of scanned Hallway output in MATLAB

Limitations:

Because this embedded system LIDAR is simpler and cheaper than a commercial LIDAR product, there are some natural limitations to both the physical implementation and the internal processing of the microprocessor of the TI board.

One of the evident limitations is the system's quantization error when converting analog signals from the TOF sensor to digital values that can be processed through the microprocessor. The resolution of the VL531X sensor is 1mm, and therefore the quantization error is equivalent to the resolution of the sensor which is 1mm, which can contribute to error of measurements taken up to 1mm variance.

Another evident limitation is the ARM cortex M4F processor's capabilities for trigonometric calculations. Generally, trigonometric calculations result in a long data type output with many decimal points of accuracy. However, in the MSP-EXP432E401Y microcontroller it can only process for single precision floating point values. The TI board includes 32x32 bit single precision registers in the Floating point unit. Because this is a smaller capability of data storage, it leads to rounding errors from the calculations between polar and cartesian coordinates. Fortunately, this is not an issue in my project design as it is designed so the coordinate conversion processes with trigonometry are done in MATLAB which is processed by a PC with much higher arithmetic capabilities.

The maximum standard serial communication rate that can be used by my PC is 128000 bps, which can be verified and found through the properties of “XDS110 Class Application/User UART” port in device manager on a PC while connected to the microcontroller. For this project, a speed of 115200 bps baud rate was used. The TOF sensor communicates through I2C communication protocol with the microcontroller at 50Hz transmission speed.

I believe the most impactful limitation was due to the stepper motor in the project, as it only correctly rotates if there is a large enough delay in between each of the four phase activations. This makes it so that the stepper motor is actually very slow, even with the lowest possible delay without stopping the motor movement. The least amount of delay for the fastest rotation was determined through testing as 2-3ms.

My appointed bus speed of 12MHz was exactly a tenth of the original max bus speed of 120MHz from studios. In order to implement this, we had to use the formula for bus frequency which is shown below:

$$\text{Bus Frequency} = 480\text{MHz} / \text{PSYDIV} + 1$$

This would mean that for a bus frequency of 12 MHz, PSYDIV would have to be equivalent to 39. The defined constant PSYDIV is found in PLL.h, where it was set as 39. Although this changes the bus frequency, in order to not affect the SysTick file configured for a 120MHz bus frequency, each function had to be adapted to the new bus speed. For example an input to SysTickWait() in SysTickWait10ms() would have to be 120000 instead of 1200000 in the SysTick.c file.

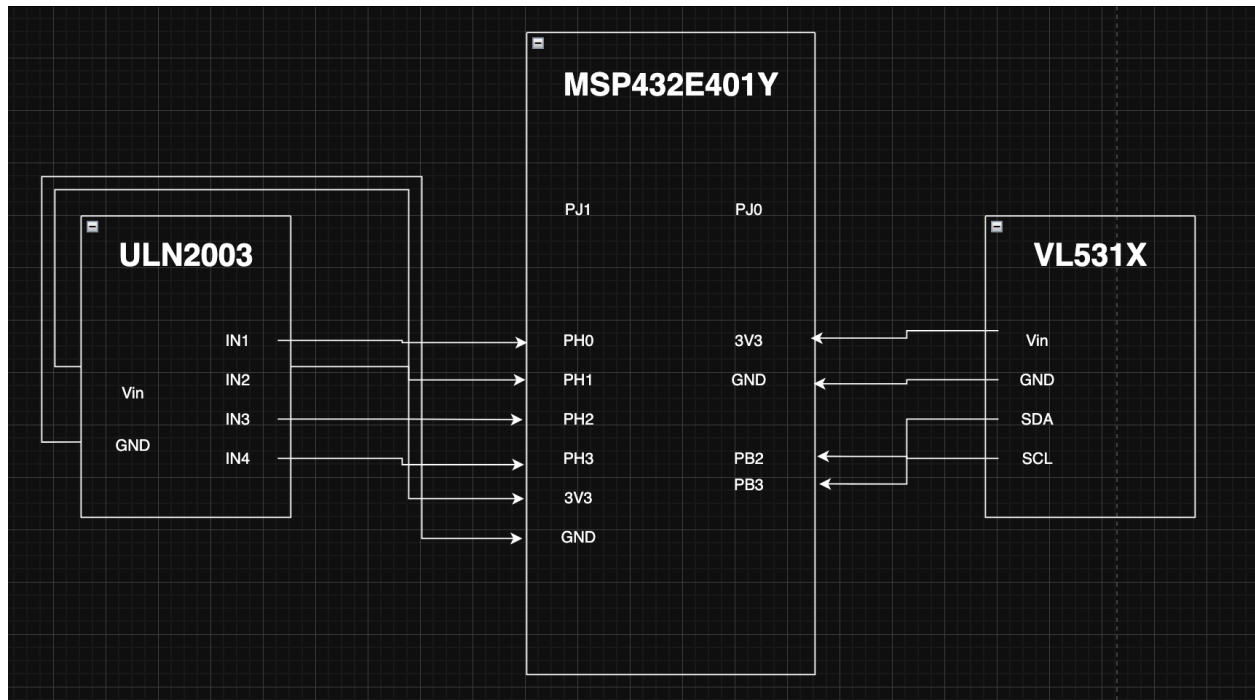


Figure 4: Circuit Schematic for Project Design

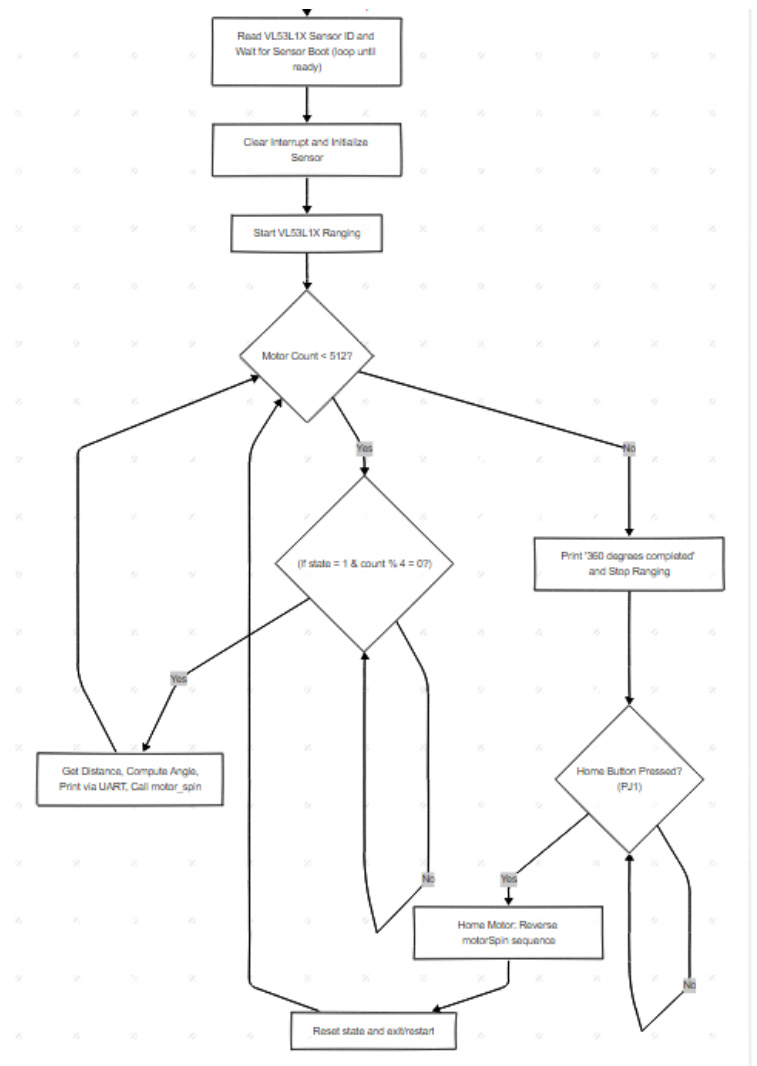
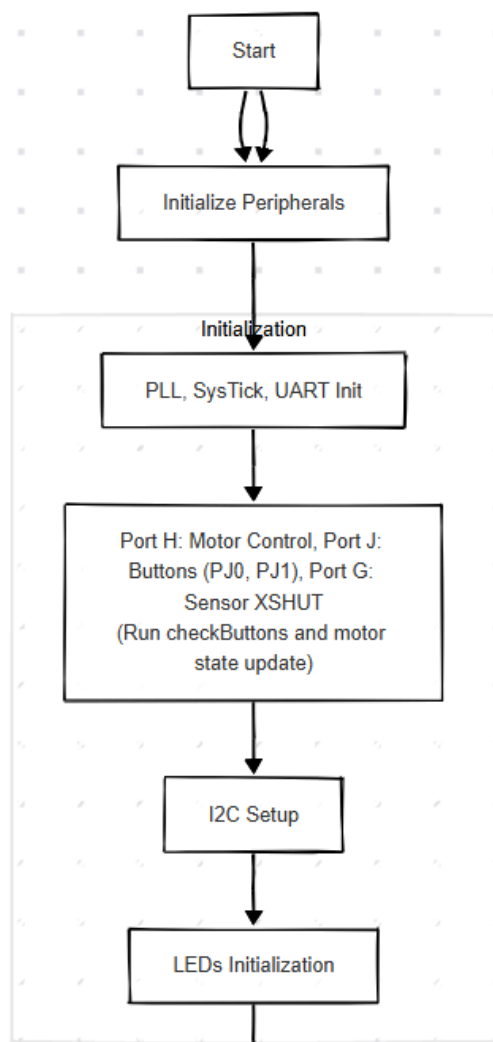


Figure 5: Program Flowchart for KEIL program, where Image 1 Bottom Connects to Image 2 Top

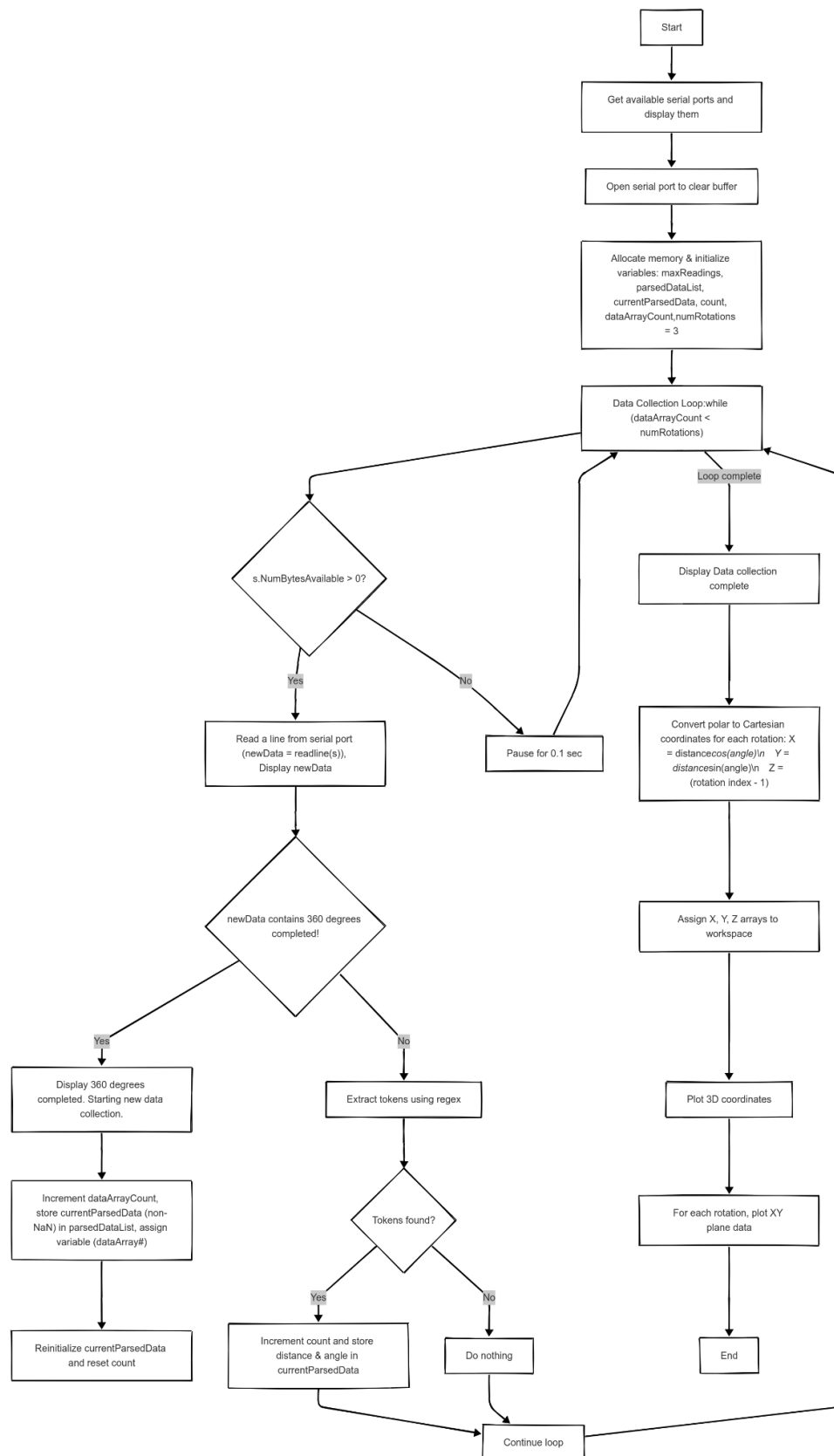


Figure 6: Program Flowchart for MATLAB program

